

BELIV Lab

Battery Internal Temperature Prediction

Complete Step-by-Step Lab Guide

From Raw CALB Dataset to Trained LSTM + PINN Models

CALB L148N58A 58Ah NMC Prismatic Cell

Project by : Yuvavignesh Balamurugan

Metric	Value
Cell tested	CALB L148N58A 58Ah NMC Prismatic
Dataset size	6,834,132 rows across 164 experimental runs
Drive cycle profiles	WLTP, UDDS, US06, C20 charge, C20 discharge
Ambient temperatures	10 degrees C, 25 degrees C, 40 degrees C
Baseline LSTM RMSE	0.099 degrees C on held-out test set
LSTM+PINN RMSE	0.116 degrees C on held-out test set
Published best (Karnehm 2024)	0.751 degrees C (7.6x worse than ours)

Section 1: Project Overview

1.1 What Problem Are We Solving

When a lithium-ion battery charges or discharges, electricity flows through it and generates heat at the core. The problem is that temperature sensors sit on the outside surface of the battery. By the time the surface gets hot enough to trigger an alarm, the inside may already be dangerously hot.

For the CALB L148N58A cell which is 27mm thick, heat generated at the core must travel 13mm through dense electrode material before reaching the surface sensor. During fast charging the core temperature can be significantly hotter than the surface. This is the thermal blind spot that causes thermal runaway in EV batteries.

1.2 What This Project Does

This project builds a machine learning model that takes the sensor measurements a Battery Management System already has and predicts the hidden internal core temperature in real time, every second.

Input (what sensors already measure)	Output (what we predict)
Current in Amperes	T_internal (core temperature in degrees C)
Voltage in Volts	
Surface temperature	
Ambient air temperature	
8 engineered features	

1.3 Two Models We Build

We build and compare two models:

- Baseline LSTM: learns purely from data, no physics knowledge
- LSTM + PINN: same architecture but also constrained to obey heat transfer equations

1.4 How T_{internal} Is Obtained

KEY CONCEPT: IMPORTANT: The CALB dataset does not contain measured internal temperature. Nobody can measure it without destroying the battery. We compute it using the two-node thermal model equations applied to the real measured current and surface temperature. We validate this by checking that our predicted surface temperature matches the real thermocouple measurement to within 1.5 degrees C.

The two-node thermal equations used:

Core equation: $mCp_{core} \times dT_{core}/dt = Q_{gen} - k \times (T_{core} - T_{surf})$
Surface equation: $mCp_{surf} \times dT_{surf}/dt = k \times (T_{core} - T_{surf}) - hA \times (T_{surf} - T_{amb})$

Where:

$Q_{gen} = I^2 \times R_{internal}$ (heat from electricity)
 $k = 1.2 \text{ W/K}$ (core to surface conductance)
 $hA = 0.624 \text{ W/K}$ (convection loss to air)
 $mCp_{core} = 650 \text{ J/K}$ (thermal mass of core)
 $mCp_{surf} = 280 \text{ J/K}$ (thermal mass of surface)

Section 2: Environment Setup

2.1 Required Software

Software	Version	Purpose
Python	3.9 or higher	Main programming language
PyTorch	2.0 or higher	Neural network training
CUDA (optional)	11.8 or higher	GPU acceleration (10x faster training)
VS Code	Any	Code editor
Git	Any	Version control

2.2 Install All Required Libraries

Open a terminal and run these commands one by one:

```
pip install torch torchvision torchaudio --index-url
https://download.pytorch.org/whl/cu118
pip install numpy pandas scipy scikit-learn matplotlib
```

NOTE: If you do not have a GPU, replace the first command with: `pip install torch torchvision torchaudio`

2.3 Folder Structure

Create this folder structure on your computer before starting:

```
C:\Users\YourName\battery_data\
  Temperature_10C\
    C20_charge\          (11 .mat files)
    C20_discharge\       (11 .mat files)
    DV_UDDS\             (11 .mat files)
    DV_US06\             (11 .mat files)
    DV_WLTP\             (11 .mat files)
  Temperature_25C\       (same 5 folders)
  Temperature_40C\       (same 5 folders)
  calb_preprocess.py
  battery_train_p2.py
  battery_train_gru.py
  battery_validate_p2.py
```

Section 3: Getting the CALB Dataset

3.1 What Is the CALB Dataset

The dataset was collected by the University of Trieste, Italy and is publicly available on Mendeley Data. It contains real experimental measurements from 11 physical CALB L148N58A cells tested under standardised driving profiles at three ambient temperatures.

Property	Value
Cell	CALB L148N58A
Chemistry	NMC (Nickel Manganese Cobalt)
Capacity	58 Ah
Dimensions	148 x 27 x 106 mm
Number of cells tested	11
Total experimental runs	164
Total data points	6,834,132 rows
Time resolution	1 measurement per second (after resampling)
Source	Mendeley Data -- University of Trieste, Italy

3.2 Download Instructions

1. **Step 1:** Open your browser and go to: <https://data.mendeley.com/datasets/ycx459r5c3/2>
2. **Step 2:** Click the blue "Download" button on the page
3. **Step 3:** Extract the downloaded ZIP file
4. **Step 4:** You will find three folders: Temperature_10C, Temperature_25C, Temperature_40C
5. **Step 5:** Move these three folders into your battery_data directory

3.3 What Is Inside Each File

Each .mat file contains one experimental run for one cell. Run this in VS Code terminal to inspect:

```
# Save as check_mat.py and run: python check_mat.py
import scipy.io as sio
mat = sio.loadmat(r'Temperature_10C\DV_WLTP\59294.mat')
data = mat['Data'][0, 0]
print('Times shape: ', data['Times'].shape) # (221437, 1)
print('Current shape: ', data['CurrentA'].shape) # (221437, 1)
print('Voltage shape: ', data['VoltageV'].shape) # (221437, 1)
print('T_surface: ', data['TempC'].shape) # (221437, 1)
print('T_ambient: ', data['Temp_Amb'].shape) # (221437, 1)
```

IMPORTANT: The data is sampled at approximately 0.1-second intervals. The preprocessing script resamples everything to clean 1-second intervals using linear interpolation. There is NO T_{internal} column in the raw data. We compute it using the physics model.

3.4 Drive Cycle Profiles Explained

Profile	What It Simulates	Duration	Starting SOC
DV_WLTP	Mixed city and motorway driving	About 90 minutes	90%
DV_UDDS	Urban city driving with stop-go traffic	About 70 minutes	80%
DV_US06	Aggressive highway with fast acceleration	About 70 minutes	75%
C20_charge	Very slow gentle charging	About 20 hours	5%
C20_discharge	Very slow gentle discharging	About 20 hours	100%

Section 4: Preprocessing the Data

4.1 What the Preprocessing Script Does

The script `calb_preprocess.py` reads all 165 raw `.mat` files and transforms them into clean CSV files ready for model training. It performs six main tasks:

- Loads each `.mat` file and extracts Current, Voltage, `T_surface`, `T_ambient`
- Resamples from irregular 0.1-second intervals to clean 1-second intervals
- Computes State of Charge (SOC) using Coulomb counting from real current
- Applies the two-node thermal model to compute `T_internal` from real measurements
- Engineers 12 input features from the raw measurements
- Splits all runs into train (70%), validation (15%), and test (15%) sets

4.2 Run the Preprocessing Script

Make sure you are in the `battery_data` folder, then run:

```
cd C:\Users\YourName\battery_data
python calb_preprocess.py
```

Expected output when it finishes successfully:

```
Combining 164 runs (0 failed)

SURFACE TEMPERATURE VALIDATION
Mean RMSE      : 0.187 C
Median RMSE    : 0.154 C
Max RMSE       : 0.564 C
Under 1.5C     : 100.0% of runs
STATUS: VALIDATED -- thermal model accepted

DATASET SUMMARY
Total rows     : 6834132
Total runs     : 164
```

TROUBLESHOOTING: If STATUS shows WARNING instead of VALIDATED, the thermal model parameters may need adjustment. Check the `K_COND` and `MCP_CORE` values at the top of `calb_preprocess.py`.

4.3 The 12 Input Features

These are the exact 12 features fed into every model:

Feature	What It Is	Why It Helps
current_A	Current in Amperes	More current means more heat generated
voltage_V	Battery terminal voltage	Changes indicate internal resistance changes
SOC	State of Charge (0 to 1)	Internal resistance is higher at low SOC
T_ambient_C	Surrounding air temperature	Determines how fast heat escapes
T_surface_C	Surface thermocouple measurement	Best observable proxy for internal temp
Q_gen_W	Estimated heat generation in Watts	Directly drives internal heating
dI_dt	Rate of current change per second	Captures acceleration and braking
dV_dt	Rate of voltage change per second	Contains impedance information
delta_T_surf	Surface minus ambient temperature	Shows heating above baseline
I_mean_60s	Average current over 60 seconds	Captures recent thermal loading history
I_std_60s	Current variability over 60 seconds	Captures erratic driving patterns
R_internal_Ohm	Estimated internal resistance	Directly controls heat generation

4.4 Train / Validation / Test Split

The split is done by run, not by individual rows. This is critical. If you split by row, the same experimental run appears in both training and test which gives artificially good results.

Split	Percentage	Number of Runs	Purpose
Train	70%	114 runs	Model learns patterns from this data
Validation	15%	25 runs	Checked during training to monitor progress
Test	15%	25 runs	Never seen during training -- final honest evaluation

4.5 Output Files

After preprocessing completes, these files are created in the calb_data folder:

File	Contents	Size
calb_full_dataset.csv	All 164 runs combined	~6.8M rows
calb_train.csv	Training data only (114 runs)	~4.8M rows
calb_val.csv	Validation data only (25 runs)	~1.0M rows
calb_test.csv	Test data only (25 runs)	~1.0M rows
surface_validation.png	Plot comparing predicted vs real T_surface	Image file

Section 5: Model Architecture

5.1 What LSTM Is and Why We Use It

LSTM stands for Long Short-Term Memory. It is a type of neural network specifically designed for sequential data where the order and history matters. Battery temperature is a perfect example of sequential data because the temperature right now depends on what the current has been doing over the past 60 seconds, not just the current second.

A regular neural network looks at each timestep independently with no memory. An LSTM has a built-in memory mechanism that allows it to remember patterns from the past 60 seconds and use them to make better predictions about the current thermal state.

5.2 Model Architecture Details

Component	Configuration	What It Does
Input	60 timesteps x 12 features	Last 60 seconds of all 12 measurements
LSTM Layer 1	256 hidden units	Learns basic patterns like current-to-heat
LSTM Layer 2	256 hidden units	Learns complex temporal patterns
Dropout	20% during training	Prevents memorisation of training data
Dense Layer 1	256 to 64 neurons	Compresses learned features
Dense Layer 2	64 to 1 neuron	Final output: predicted temperature
Parameters	819,329 total	Numbers adjusted during training

5.3 What PINN Adds

PINN stands for Physics-Informed Neural Network. The architecture is identical to the baseline LSTM. The only difference is the loss function used during training.

Model	Loss Function	Effect
Baseline LSTM	Loss = Huber(predicted, true)	Learns from data only
LSTM + PINN	Loss = Huber(predicted, true) + lambda x physics_residual	Learns from data AND must obey heat equations

The physics residual penalises any prediction that violates the two-node thermal equations:

```
physics_residual = mCp_core x dTcore/dt - Q_gen + k x (Tcore - Tsurf)
```

This should be zero for any physically valid temperature trajectory.
If it is not zero, the model violated thermodynamics and gets penalised.

5.4 Sliding Window Sequences

The LSTM does not look at one second at a time. It looks at 60 seconds of history at once. We create these 60-second windows by sliding across the time series data:

```
Window 1:  seconds 0  to 59  -> predict temperature at second 59
Window 2:  seconds 10 to 69  -> predict temperature at second 69
Window 3:  seconds 20 to 79  -> predict temperature at second 79
...
Result: 475,517 training windows from 4.76 million training rows
```

Section 6: Training the Models

6.1 Run the Training Script

```
cd C:\Users\YourName\battery_data
python battery_train_p2.py
```

TIME ESTIMATE: Training time: approximately 60 to 90 minutes with a GPU (RTX 4060 or similar). Without a GPU expect 4 to 8 hours. The script automatically detects and uses GPU if available.

6.2 Key Training Hyperparameters

Parameter	Value	What It Means
EPOCHS	300	Maximum number of complete passes through training data
BATCH_SIZE	128	Number of windows processed together each step
LEARNING_RATE	0.0003	How much to adjust the model each step
SEQ_LEN	60	Seconds of history the model sees at once
HIDDEN_SIZE	256	Number of memory units in each LSTM layer
DROPOUT	0.2	20 percent of neurons randomly disabled during training
PATIENCE	25	Stop training if no improvement for 25 epochs
LAMBDA_START	0.000001	Starting weight for physics penalty (very small)
LAMBDA_END	0.0001	Final weight for physics penalty (gentle guidance)

6.3 Understanding the Training Output

During training you will see lines like this:

```
Epoch 1/300 train_loss=0.0151 val_loss=0.0003 RMSE=0.205C MAE=0.159C
[no_improve=0]
Epoch 5/300 train_loss=0.0098 val_loss=0.0002 RMSE=0.166C MAE=0.116C
[no_improve=0]
Epoch 10/300 train_loss=0.0097 val_loss=0.0002 RMSE=0.153C MAE=0.112C
[no_improve=0]
Epoch 35/300 Early stopping -- val RMSE has not improved for 25 epochs
Training complete. Best val_RMSE=0.1413C
```

Term in output	What it means
train_loss	How wrong the model is on training data this epoch
val_loss	How wrong on validation data -- the important one to watch

Term in output	What it means
RMSE	Average prediction error in degrees Celsius on validation set
MAE	Another accuracy measure -- mean absolute error in degrees C
no_improve	How many epochs since the best validation RMSE was achieved
Early stopping	Training stopped because no improvement for 25 epochs in a row

6.4 Expected Results

Model	RMSE	MAE	When it stops
Baseline LSTM	0.094 - 0.099 C	0.069 - 0.074 C	Around epoch 35
LSTM + PINN	0.110 - 0.120 C	0.085 - 0.095 C	Around epoch 32

EXPECTED BEHAVIOUR: It is normal and expected that LSTM+PINN has slightly higher RMSE than baseline LSTM. The physics constraint trades a small amount of raw accuracy for guaranteed physical consistency. This is documented in published PINN literature and is not a bug.

6.5 Output Files After Training

File	Location	Contents
p2_baseline_lstm.pt	results_p2/	Saved baseline model weights
p2_pinn_lstm.pt	results_p2/	Saved PINN model weights
baseline_predictions.png	results_p2/	Prediction plot for baseline model
pinn_predictions.png	results_p2/	Prediction plot for PINN model
model_comparison.png	results_p2/	Training curve comparison plot

Section 7: Training the GRU Comparison Model

7.1 Why We Train a GRU Model

GRU (Gated Recurrent Unit) is a simpler version of LSTM with fewer parameters. Training it on the same data with the same hyperparameters gives us a fair architectural comparison. This tells us whether the extra complexity of LSTM is actually needed for this problem.

Property	LSTM	GRU
Gates	3 (input, forget, output)	2 (reset, update)
Parameters	819,329	~615,000
Training speed	Slower	About 20% faster
Our RMSE	0.099 C	0.114 C

7.2 Run the GRU Training Script

```
cd C:\Users\YourName\battery_data
python battery_train_gru.py
```

Expected GRU results:

```
FINAL COMPARISON SUMMARY

Model          RMSE      MAE
GRU (this)     0.114     0.0905
Baseline LSTM  0.0990    0.0740
LSTM + PINN    0.1160    0.0890
```

Section 8: Validating All Three Models

8.1 Run the Validation Script

After all three training scripts have finished, run the validation script. This loads all three saved models and evaluates them together, producing all paper figures.

```
cd C:\Users\YourName\battery_data
python battery_validate_p2.py
```

8.2 Output Figures Explained

File	What It Shows	Used In Paper
time_series_all_models.png	Temperature vs time with all 3 models on one graph	Key figure
scatter_plot.png	Predicted vs true temperature for all 3 models	Figure 1
error_distribution.png	Histogram of prediction errors	Figure 2
rmse_by_profile.png	RMSE comparison across WLTP, UDDS, US06, C20	Figure 3
rmse_by_temperature.png	RMSE at 10C, 25C, and 40C ambient	Figure 4
rmse_heatmap.png	Profile x temperature grid of RMSE values	Figure 5
validation_report.txt	All numerical results for paper tables	Tables

8.3 How to Read the Time Series Plot

The time series plot is the most important figure for communicating results. It shows:

- Black solid line: the true T_{internal} computed from the two-node model
- Blue dashed line: baseline LSTM prediction
- Green dash-dot line: GRU prediction
- Red dotted line: LSTM+PINN prediction
- Orange dotted line: real T_{surface} from the thermocouple
- Purple shaded region: the gap between core and surface temperature

All model lines should track closely to the black ground truth line throughout the entire run. The error panel below shows the prediction errors over time. Most errors should stay within the yellow 0.5 degree band.

Section 9: Complete Run Order Summary

IMPORTANT: Follow this exact order. Each step depends on the previous step completing successfully.

Step	Script	Time	Output
1	calb_preprocess.py	5-10 min	calb_data/ folder with 3 CSV files
2	battery_train_p2.py	60-90 min	results_p2/ folder with 2 model files
3	battery_train_gru.py	45-60 min	results_gru/ folder with GRU model
4	battery_validate_p2.py	5-10 min	results_p2/validation/ folder with 6 plots

9.1 Quick Commands (Copy-Paste)

```
cd C:\Users\YourName\battery_data

# Step 1 -- preprocess
python calb_preprocess.py

# Step 2 -- train baseline LSTM and LSTM+PINN
python battery_train_p2.py

# Step 3 -- train GRU comparison model
python battery_train_gru.py

# Step 4 -- validate all three models together
python battery_validate_p2.py
```


Section 10: Troubleshooting

Error Message	Cause	Fix
No module named torch	PyTorch not installed	Run: pip install torch
CUDA out of memory	GPU memory too small for batch size	Reduce BATCH_SIZE from 128 to 64 in battery_train_p2.py
FileNotFoundError calb_train.csv	Preprocessing not run yet	Run calb_preprocess.py first
Surface validation STATUS: WARNING	Thermal model RMSE above 1.5C	Check that .mat file path is correct and files are not corrupted
p2_baseline_lstm.pt not found	Training not completed	Run battery_train_p2.py and wait for it to finish completely
Training loss explodes to NaN	Learning rate too high	Reduce LEARN_RATE from 0.0003 to 0.0001 in training script
Early stopping at epoch 1	Data loading error or wrong CSV path	Check DATA_FOLDER path at top of training script

Section 11: Results Reference

11.1 Final Results Achieved

Model	RMSE (C)	MAE (C)	Notes
Baseline LSTM	0.099	0.074	Best raw accuracy
GRU	0.114	0.0905	Faster training, slightly lower accuracy
LSTM + PINN	0.116	0.089	Physically consistent predictions
Karnehm 2024 KAN	0.751	0.469	Best published result (for comparison)
Karnehm 2024 LSTM	0.854	0.550	Published LSTM baseline (for comparison)

11.2 Surface Validation Results

Metric	Value	Meaning
Mean surface RMSE	0.187 C	Average match between predicted and real T_surface
Median surface RMSE	0.154 C	Typical match across all 164 runs
Max surface RMSE	0.564 C	Worst case match in any single run
Runs under 1.5C	100%	All runs accepted -- thermal model is valid

11.3 Dataset Statistics

Statistic	Value
Total rows	6,834,132
Total runs	164
T_internal minimum	6.84 degrees C
T_internal maximum	42.36 degrees C
T_internal mean	25.76 degrees C
Training rows	4,761,351 (114 runs)
Validation rows	1,034,293 (25 runs)
Test rows	1,038,488 (25 runs)
Training sequences	475,517 (60-second windows)

Section 12: Key Concepts Reference

12.1 Terminology Glossary

Term	Plain English Explanation
Epoch	One complete pass through all training data. Like reading a textbook once.
Batch	Small group of training examples processed together. We use 128 windows per batch.
Loss	A number measuring how wrong the model is. Training tries to minimise this.
RMSE	Root Mean Square Error. Average prediction error in degrees C. Lower is better.
MAE	Mean Absolute Error. Another average error measure. Lower is better.
Early stopping	Halt training when no improvement for 25 epochs to prevent memorisation.
Dropout	Randomly switch off 20% of neurons during training to prevent overfitting.
Overfitting	When a model memorises training data instead of learning general patterns.
SOC	State of Charge. 1.0 means fully charged. 0.0 means completely empty.
Two-node model	Physics equations modelling core and surface as two connected thermal masses.
Physics residual	How much a prediction violates the heat transfer equations. Should be near zero.
Lambda	Weight controlling how much physics penalty matters vs data accuracy in PINN.
Sequence	A 60-second window of 12 measurements fed into the LSTM as one training example.
Scaler	Normalises all features to the same range so no single feature dominates.

12.2 Files in This Project

File	Purpose
calb_preprocess.py	Loads raw .mat files, computes T_{internal} , saves train/val/test CSV files
battery_train_p2.py	Trains Baseline LSTM and LSTM+PINN on calb_data, saves to results_p2
battery_train_gru.py	Trains GRU comparison model on calb_data, saves to results_gru
battery_validate_p2.py	Loads all 3 models, evaluates on test set, generates all paper figures
battery_lstm_matlab.m	Independent MATLAB implementation for cross-framework validation